CPSC 304 Project Cover Page

Milestone #: ___4____

Date: _03/04/2025__

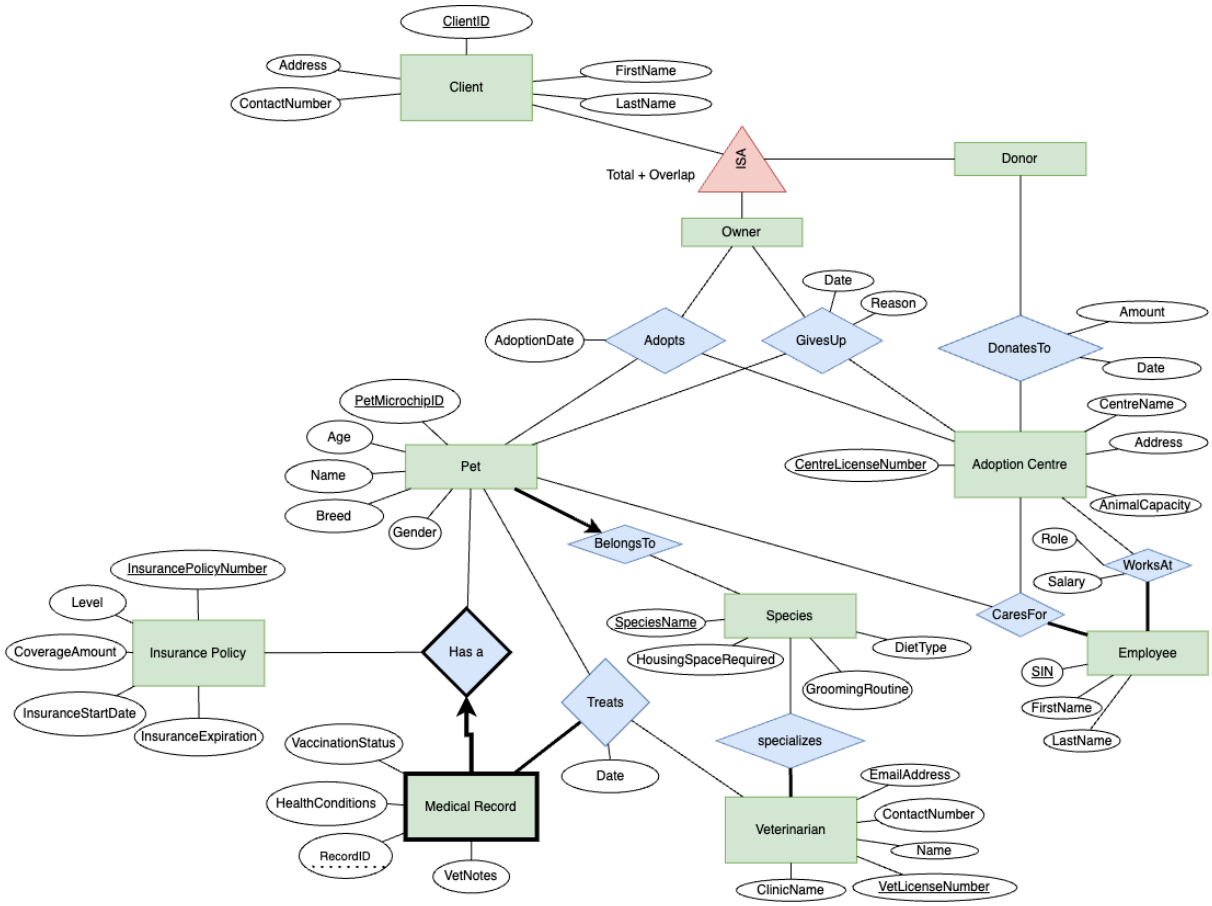
Group Number: ___92___

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Krish Arora	17516238	w0a4z	karora07@student.ubc.ca
Jaskarn Nijjar	74883810	a9b4j	jaskarn.nijjar75@gmail.com
Sophie di Vittorio	24858961	y4b7k	sophiedivittorio@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

This project allows users to track information about pets added to the database, and the various relevant resources for their care. Depending on the role of the user, they can add a pet with some details, view/add veterinary clinics and animal shelters, animal medical histories, adoption history, and species care instructions. The user can then select and filter for certain features based on their needs. We have not added login requirements in this project as it is beyond the scope of the project, but the idea would that different users can alter different aspects of the database is reflected in the links available for the type of user seen in the main menu.



The final schema was changed in that, while you can register as a donor, we did not add the DonatesTo relation as it did not contribute meaningfully to the database information that we ended up implementing - we ended up focusing more on the pets and their relevant information. Otherwise, the schema remains the same.

Queries

1. Insert

Relations needed:

- You must choose one relation to implement INSERT on.

Input format:

- The user should be able to specify what values to insert.

Users may insert a new pet, account, vet practice, shelter.

```
async function insertNewPet(MicrochipID, Name, Age, Breed, Gender, SpeciesName) {
    return await withOracleDB(async (connection) => {
        const result = await connection.execute(
            `INSERT INTO Pet (PetMicrochipID, Name, Age, Breed, Gender, SpeciesName) VALUES
            (:MicrochipID, :Name, :Age, :Breed, :Gender, :SpeciesName)`,
            [MicrochipID, Name, Age, Breed, Gender, SpeciesName],
            { autoCommit: true }
        );

        return result.rowsAffected && result.rowsAffected > 0;
    }).catch(() => {
        return false;
    });
}
```

2. Update

Users may update Client information

```
async function updateClient(clientId, FirstName, LastName, ClientAddress, ClientContact) {
    return await withOracleDB(async (connection) => {
        const result = await connection.execute(
            `UPDATE Client
             SET FirstName = :FirstName,
                 LastName = :LastName,
                 ClientAddress = :ClientAddress,
                 ClientContact = :ClientContact
             WHERE ClientID = :clientId`,
            [FirstName, LastName, ClientAddress, ClientContact, clientId],
            { autoCommit: true }
        );
        return result.rowsAffected && result.rowsAffected > 0;
    }).catch(() => {
        return false;
    });
}
```

3. Delete

Users may

```
async function deleteSpecies(speciesName) {
    return await withOracleDB(async (connection) => {
      await connection.execute(
        `DELETE FROM Pet WHERE SpeciesName = :speciesName`,
        [speciesName],
        { autoCommit: true }
      );
      const result = await connection.execute(
        `DELETE FROM Species WHERE speciesName = :speciesName`,
        [speciesName],
        { autoCommit: true }
      );
      return result.rowsAffected && result.rowsAffected > 0;
    }).catch((error) => {
      console.error("Error deleting species:", error);
      return false;
    });
  }
```

4. Selection

Users may select a specific species and/or age of pet to view

```javascript
async function fetchPetTableFromDb(species, minAge, maxAge) {
    return await withOracleDB(async (connection) => {
        let query = 'SELECT * FROM Pet';
        let conditions = [];
        let params = {};

        if (species) {
            conditions.push('SpeciesName = :species');
            params.species = species;
        }

        if (minAge) {
            conditions.push('Age >= :minAge');
            params.minAge = minAge;
        }

        if (maxAge) {
            conditions.push('Age <= :maxAge');
            params.maxAge = maxAge;
        }

        if (conditions.length > 0) {
            query += ' WHERE ' + conditions.join(' AND ');
        }

        const result = await connection.execute(query, params);
        return result.rows;
    }).catch((error) => {
        console.error("Database error:", error);
        return [];
    });
}
```

5. Projection

Users may choose to view any number of details about Vet clinics

```javascript
async function fetchVetProject(selectors) {
    return await withOracleDB(async (connection) => {
        const text = "SELECT " + selectors + " FROM Veterinarian"
        const result = await connection.execute(text);
        return result.rows;
    }).catch(() => {
        return [];
    });
}
```

“selectors” describes the details selected by checkbox

6. Join

Relations needed:

- You must implement one query of this category.
- This query must join at least two relations of your choice.

Input format:

- The user must provide at least one value to qualify in the WHERE clause (e.g., join the Customer and the Transaction relation to find the names and phone numbers of all customers *who have purchased a specific item*).

X

7. Aggregation with GROUP BY

Relations needed:

- You must implement one query of this category using relation(s) of your choice.

Users can see the average age of pets by species

```
async function fetchPetMaxAges() {
    return await withOracleDB(async (connection) => {
        const result = await connection.execute('SELECT SpeciesName, AVG(Age) FROM Pet GROUP BY(SpeciesName)');
        return result.rows;
    }).catch(() => {
        return [];
    });
}
```

8. Aggregation with HAVING

X

9. Nested aggregation with GROUP BY

Relations needed:

- You must implement one query of this category using relation(s) of your choice.

X

10.Division

Relations needed:

- You must implement one query of this category using relation(s) of your choice.

x